Overview Tasks Solutions

Not Running

For the best experience, choose the region closest to you. Then, start the lab to begin.

MARK UNFINISHED

Select the region closest to you

US-East

Keyboard layout: US English (US)

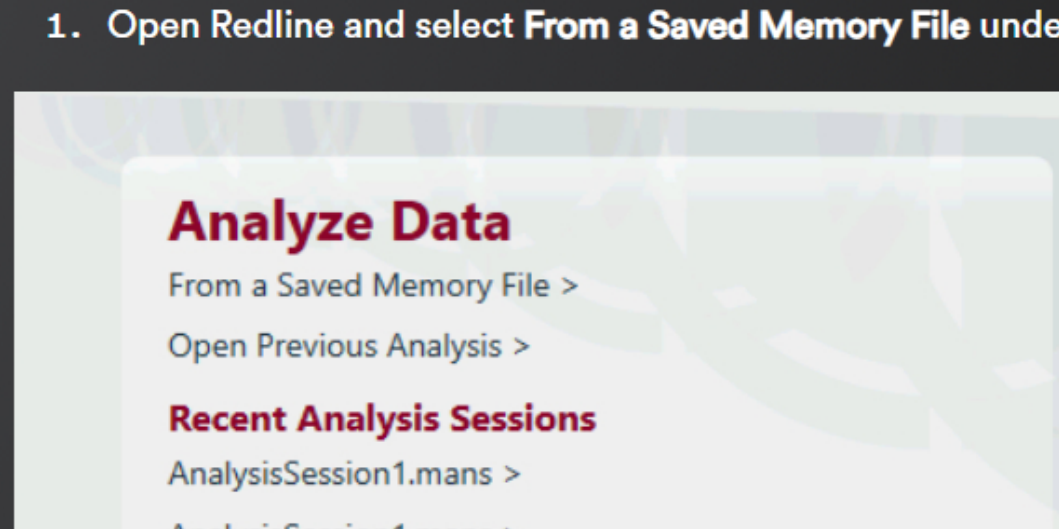
Start lab

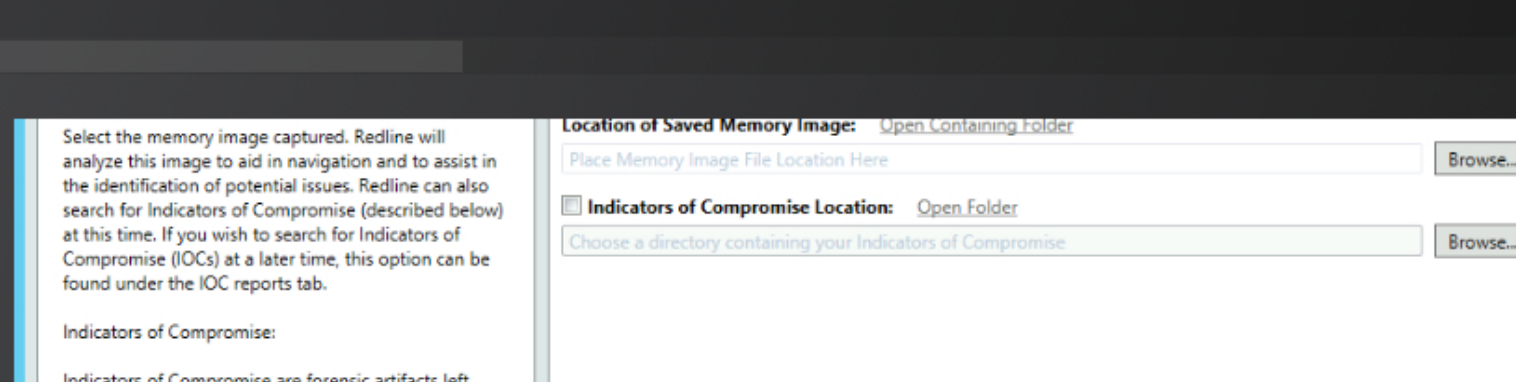
SOLUTIONS

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1: Create a MANs file of APTA.VMEM.

1. Open Redline and select **From a Saved Memory File** under **Analyze Data**.



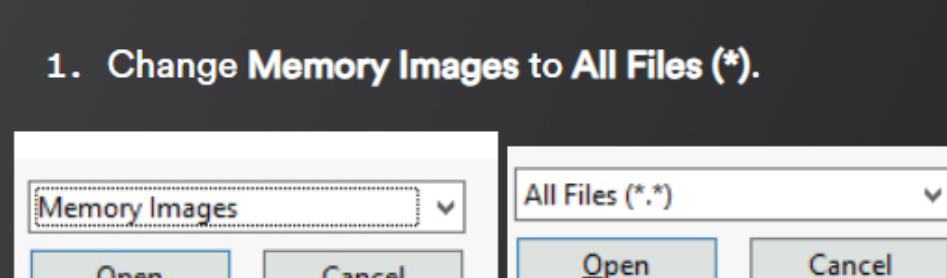

Select the memory image captured. Redline will analyze this image to aid in navigation and to assist in the identification of potential issues. Redline can also search for Indicators of Compromise (described below) at this time. If you wish to search for Indicators of Compromise (IOCs) at a later time, this option can be found under the IOC reports tab.

Indicators of Compromise:

Indicators of Compromise are forensic artifacts left behind by an intrusion. An IOC file describes these

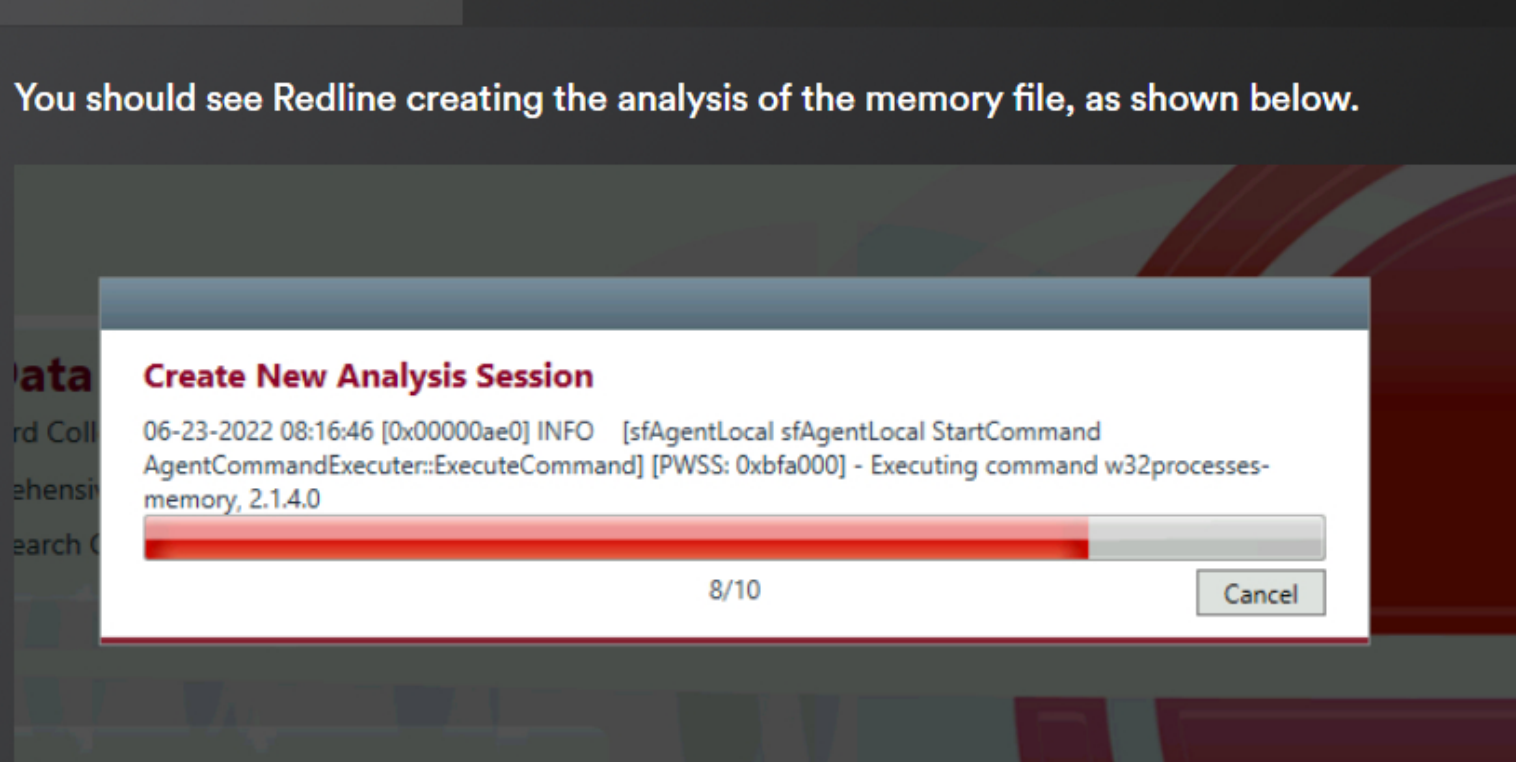
Location of Saved Memory Image: Open Containing Folder
Place Memory Image File Location Here
Browse...
☐ Indicators of Compromise Location: Open Folder
Choose a directory containing your Indicators of Compromise
Browse...

1. Change **Memory Images** to **All Files (*)**.



1. Choose **apta.vmem** and choose **Next**.
2. Click **Edit your script** select **Strings** for both **Process Listing** and **Drivers Enumeration** then choose **Ok**.
3. Change the **Name** to **APTA** and choose **Ok**.
4. If UAC is enabled, select **Yes**.

You should see Redline creating the analysis of the memory file, as shown below.


Create New Analysis Session

06-23-2022 08:16:46 [0x00000ae0] INFO [sfAgentLocal sfAgentLocal StartCommand AgentCommandExecuter::ExecuteCommand] [PWSS: 0xbfa000] - Executing command w32processes-memory, 2.1.4.0

Task 2: Analyze the MANS file from task 1.

1. After analysis is complete, we can look at the processes by selecting **Processes** under **Analysis Data**.

As we can see nothing is redlined. Based on what we know about Windows processes we can attempt to see if anything is odd (wrong parent process, wrong path, etc.). To do this we can switch to **Hierarchical Processes** view.

1. Click on **Hierarchical Processes** view under **Analysis Data**.

Do we see anything out of the ordinary? If not then let's look at **Memory Sections** for signs of code injection.

1. Click on **Memory Sections** (submenu item under **Processes**) under **Analysis Data**.
2. Click on **Injected Memory Sections** under **Filters**.

We should see 24 items listed as injected with code and by glancing at the results it appears to be nearly every processes within the memory image.

We can look at **Strings** and/or **Ports** but instead we'll look at **Hooks** to see any indicators of some sort of rootkit.

1. Click on **Hooks** under **Analysis Data**.
2. Click on **SSDT Hooks** under **Filters**.
3. Click on **IDT Hooks** under **Filters**.
4. Click on **Untrusted Hooks** under **Filters**.

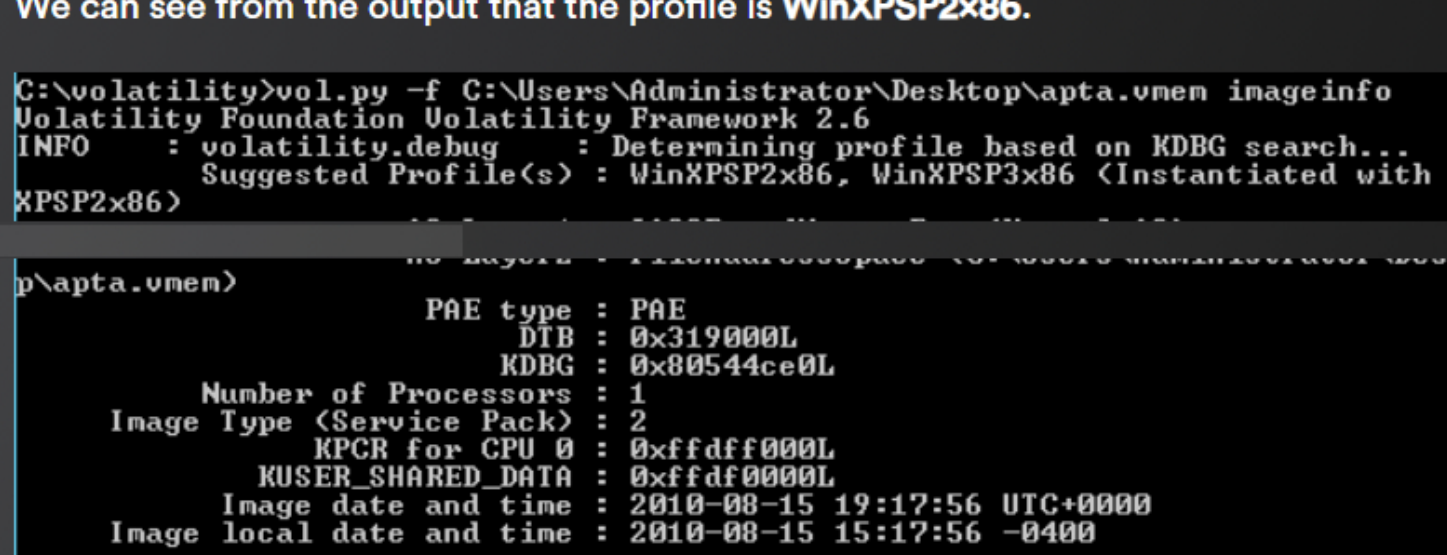
We didn't see anything under SSDT and IDT Hooks but we're seeing some entries under Untrusted Hooks. At this point, with the data gathered, we definitely know there was code injection in many processes. We can now take a closer look at things in Volatility.

Task 3: Analyze APTA.VMEM within Volatility.

Copy paste the apta.vmem on desktop

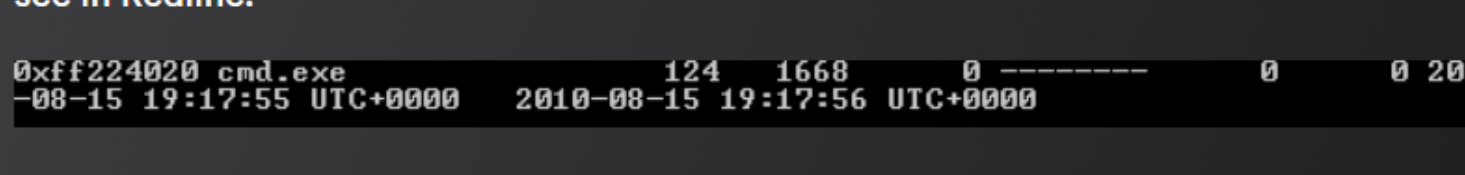
```
# We will need to determine what profile to use for this memory image file within V
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem imageinfo
```

We can see from the output that the profile is **WinXPSP2x86**.


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\apta.vmem imageinfo
Volatility Foundation Volatility Framework 2.6
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : WinXPSP2x86, WinXPSP3x86 (Instantiated with Win
XPSP2x86)
p\apta.vmem)
                     PAE type : PAE
                          DTB : 0x319000L
                         KDBG : 0x80544ce0L
         Number of Processors : 1
    Image Type (Service Pack) : 2
               KPCR for CPU 0 : 0xffdff000L
            KUSER_SHARED_DATA : 0xffdf0000L
          Image date and time : 2010-08-15 19:17:56 UTC+0000
    Image local date and time : 2010-08-15 15:17:56 -0400
```

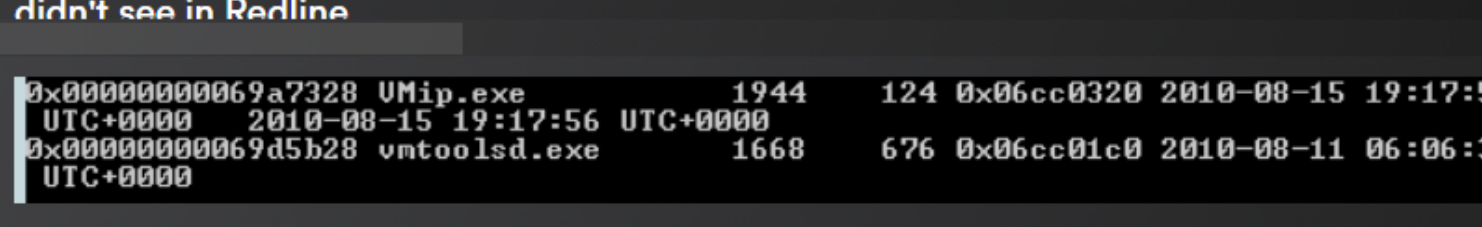
```
# We'll begin by looking at the processes with Volatility using PSLIST.
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 p
```

If we compare Redline and Volatility we should see 1 process, cmd.exe (124), that we didn't see in Redline.


```
0xff224020 cmd.exe                124   1668      0 --------      0      0 2010
-08-15 19:17:55 UTC+0000   2010-08-15 19:17:56 UTC+0000
```

```
# Next we can take a look at the processes with Volatility using PSSCAN.
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86 p
```

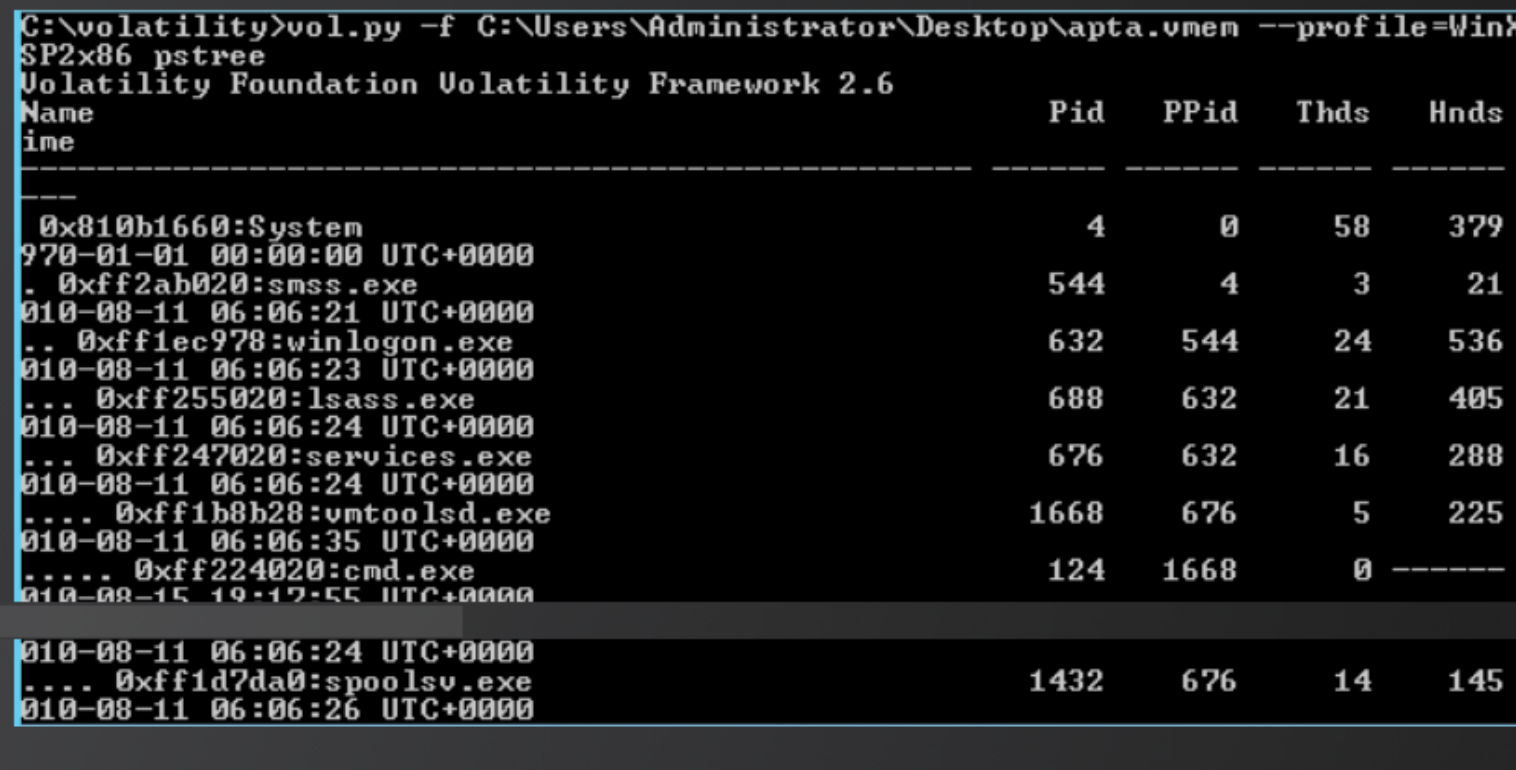
If we compare Redline and Volatility we should see another process, VMip.exe (1944), that we didn't see in Redline.


```
0x00000000069a7328 VMip.exe            1944    124 0x06cc0320 2010-08-15 19:17:55
 UTC+0000   2010-08-15 19:17:56 UTC+0000
0x00000000069d5b28 vmtoolsd.exe        1668    676 0x06cc01c0 2010-08-11 06:06:35
 UTC+0000
```

This process looks like it's a child process to cmd.exe (124).

We can use the Volatility **PSTREE** plugin to get a hierarchical process.

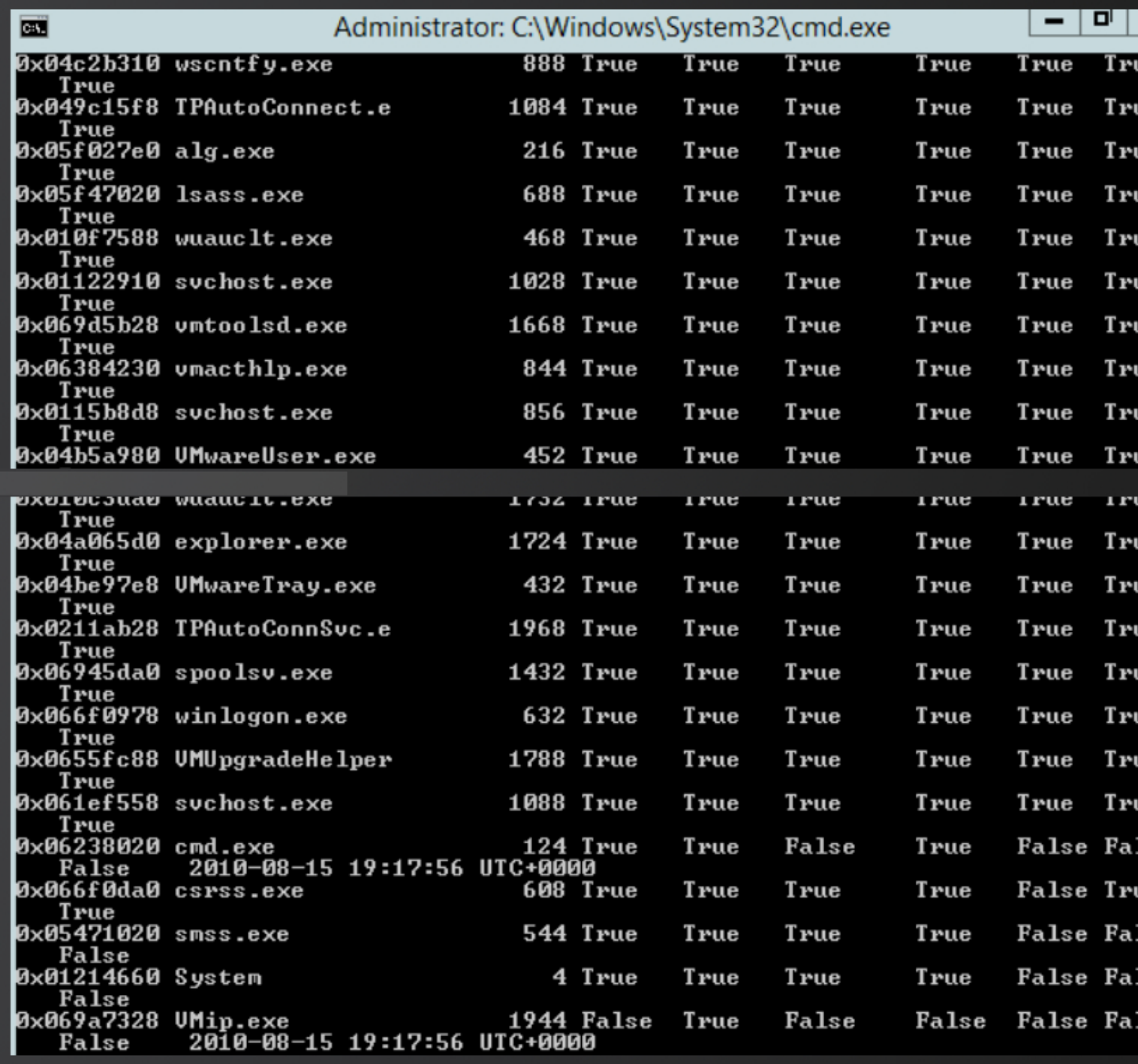
```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86
```


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXP
SP2x86 pstree
Volatility Foundation Volatility Framework 2.6
Name                                                  Pid   PPid   Thds   Hnds T
ime
-------------------------------------------------- ------ ------ ------ ------ -
---
 0x810b1660:System                                      4      0     58    379 1
970-01-01 00:00:00 UTC+0000
. 0xff2ab020:smss.exe                                 544      4      3     21 2
010-08-11 06:06:21 UTC+0000
.. 0xff1ec978:winlogon.exe                            632    544     24    536 2
010-08-11 06:06:23 UTC+0000
... 0xff255020:lsass.exe                              688    632     21    405 2
010-08-11 06:06:24 UTC+0000
... 0xff247020:services.exe                           676    632     16    288 2
010-08-11 06:06:24 UTC+0000
.... 0xff1b8b28:vmtoolsd.exe                         1668    676      5    225 2
010-08-11 06:06:35 UTC+0000
..... 0xff224020:cmd.exe                              124   1668      0 ------ 2
010-08-11 06:06:24 UTC+0000
.... 0xff1d7da0:spoolsv.exe                          1432    676     14    145 2
010-08-11 06:06:26 UTC+0000
```

From the output we can't see VMip.exe (1944) which was spawned by cmd.exe (124) but we can see cmd.exe (124).

We can use **PSXVIEW** to see if this process is showing indicators that it's a hidden process.

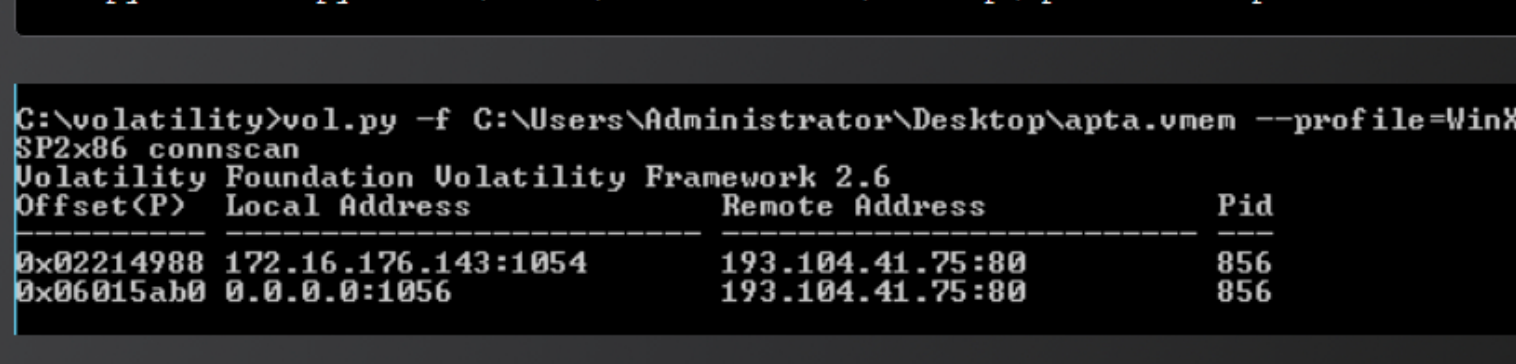
```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vem --profile=WinXPSP2x86 p
```


Administrator: C:\Windows\System32\cmd.exe

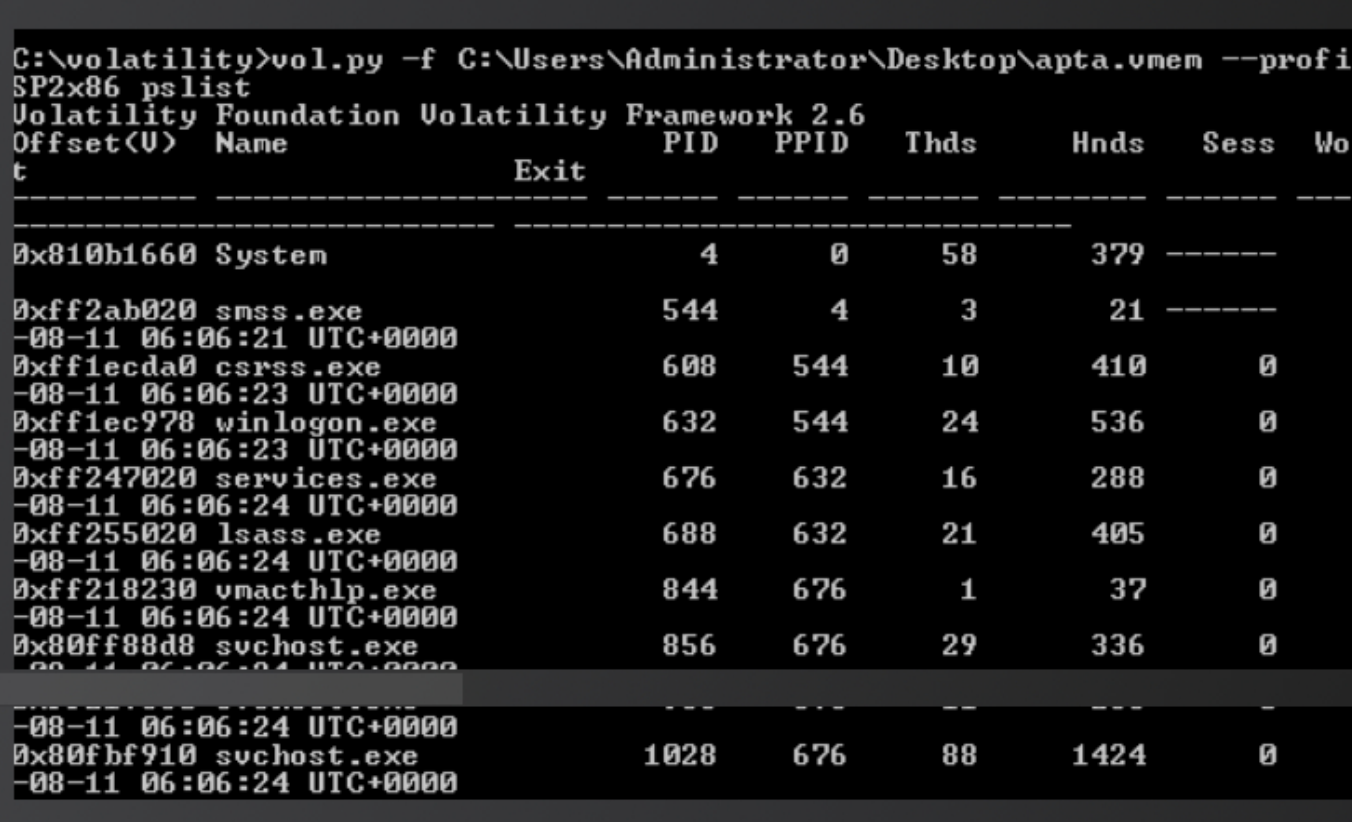
```
0x04c2b310 wscntfy.exe             888 True   True   True    True   True   True
   True
0x049c15f8 TPAutoConnect.e        1084 True   True   True    True   True   True
   True
0x05f027e0 alg.exe                 216 True   True   True    True   True   True
   True
0x05f47020 lsass.exe               688 True   True   True    True   True   True
   True
0x010f7588 wuauclt.exe             468 True   True   True    True   True   True
   True
0x01122910 svchost.exe            1028 True   True   True    True   True   True
   True
0x069d5b28 vmtoolsd.exe           1668 True   True   True    True   True   True
   True
0x06384230 vmacthlp.exe            844 True   True   True    True   True   True
   True
0x0115b8d8 svchost.exe             856 True   True   True    True   True   True
   True
0x04b5a980 VMwareUser.exe          452 True   True   True    True   True   True
   True
0x04a065d0 explorer.exe           1724 True   True   True    True   True   True
   True
0x04be97e8 VMwareTray.exe          432 True   True   True    True   True   True
   True
0x0211ab28 TPAutoConnSvc.e        1968 True   True   True    True   True   True
   True
0x06945da0 spoolsv.exe            1432 True   True   True    True   True   True
   True
0x066f0978 winlogon.exe            632 True   True   True    True   True   True
   True
0x0655fc88 VMUpgradeHelper        1788 True   True   True    True   True   True
   True
0x061ef558 svchost.exe            1088 True   True   True    True   True   True
   True
0x06238020 cmd.exe                 124 True   True   False   True   False False
   False     2010-08-15 19:17:56 UTC+0000
0x066f0da0 csrss.exe               608 True   True   True    True   False True
   True
0x05471020 smss.exe                544 True   True   True    True   False False
   False
0x01214660 System                    4 True   True   True    True   False False
   False
0x069a7328 VMip.exe               1944 False  True   False   False  False False
   False     2010-08-15 19:17:56 UTC+0000
```

By the output from **PSXVIEW** we see that VMip.exe (1944) shows indicators of a hidden process, along with cmd.exe (124). Remember that we didn't see these 2 processes in Redline but does this mean that these 2 processes are malicious? Remember that Redline indicated many processes were injected except these 2 processes because they were not visible in Redline. Let's see if VMip.exe is trying to connect to any remote IP addresses by using **CONNSCAN** (Windows XP).

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86
```


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXP
SP2x86 connscan
Volatility Foundation Volatility Framework 2.6
Offset(P)  Local Address             Remote Address            Pid
---------- ------------------------- ------------------------- ---
0x02214988 172.16.176.143:1054       193.104.41.75:80          856
0x06015ab0 0.0.0.0:1056              193.104.41.75:80          856
```

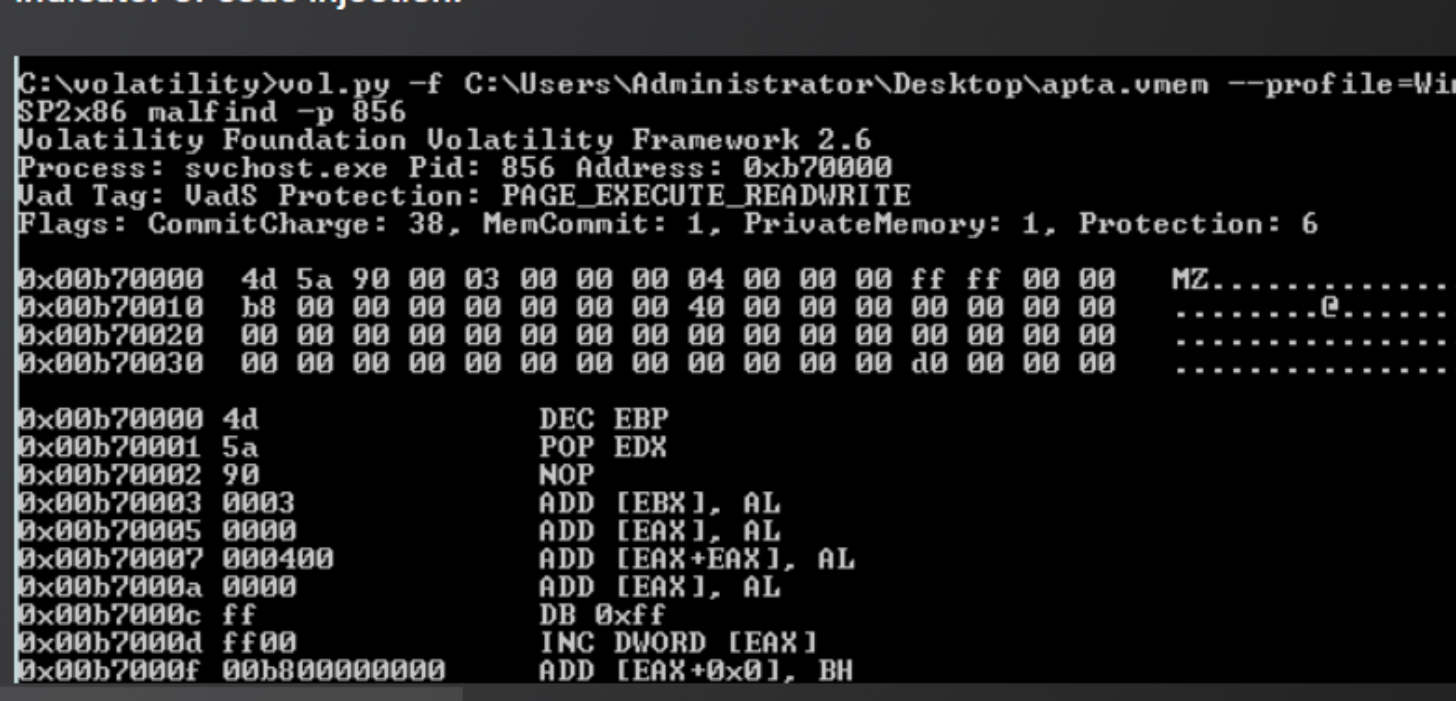
From the output we can see that process 856 is made outbound connections on ports 1054 & 1056 to an IP on port 80. What is process 856? The answer is svchost.exe.


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=
SP2x86 pslist
Volatility Foundation Volatility Framework 2.6
Offset(V)  Name                    PID   PPID   Thds     Hnds   Sess  Wow64
t                          Exit
---------- -------------------- ------ ------ ------ -------- ------ ------
------------------------- ------------------------------
0x810b1660 System                    4      0     58      379 ------      0

0xff2ab020 smss.exe                544      4      3       21 ------      0
-08-11 06:06:21 UTC+0000
0xff1ecda0 csrss.exe               608    544     10      410      0      0
-08-11 06:06:23 UTC+0000
0xff1ec978 winlogon.exe            632    544     24      536      0      0
-08-11 06:06:23 UTC+0000
0xff247020 services.exe            676    632     16      288      0      0
-08-11 06:06:24 UTC+0000
0xff255020 lsass.exe               688    632     21      405      0      0
-08-11 06:06:24 UTC+0000
0xff218230 vmacthlp.exe            844    676      1       37      0      0
-08-11 06:06:24 UTC+0000
0x80ff88d8 svchost.exe             856    676     29      336      0      0
-08-11 06:06:24 UTC+0000
0x80fbf910 svchost.exe            1028    676     88     1424      0      0
-08-11 06:06:24 UTC+0000
```

This process is one of the processes shown as injected. Let's us **MALFIND** to further examine.

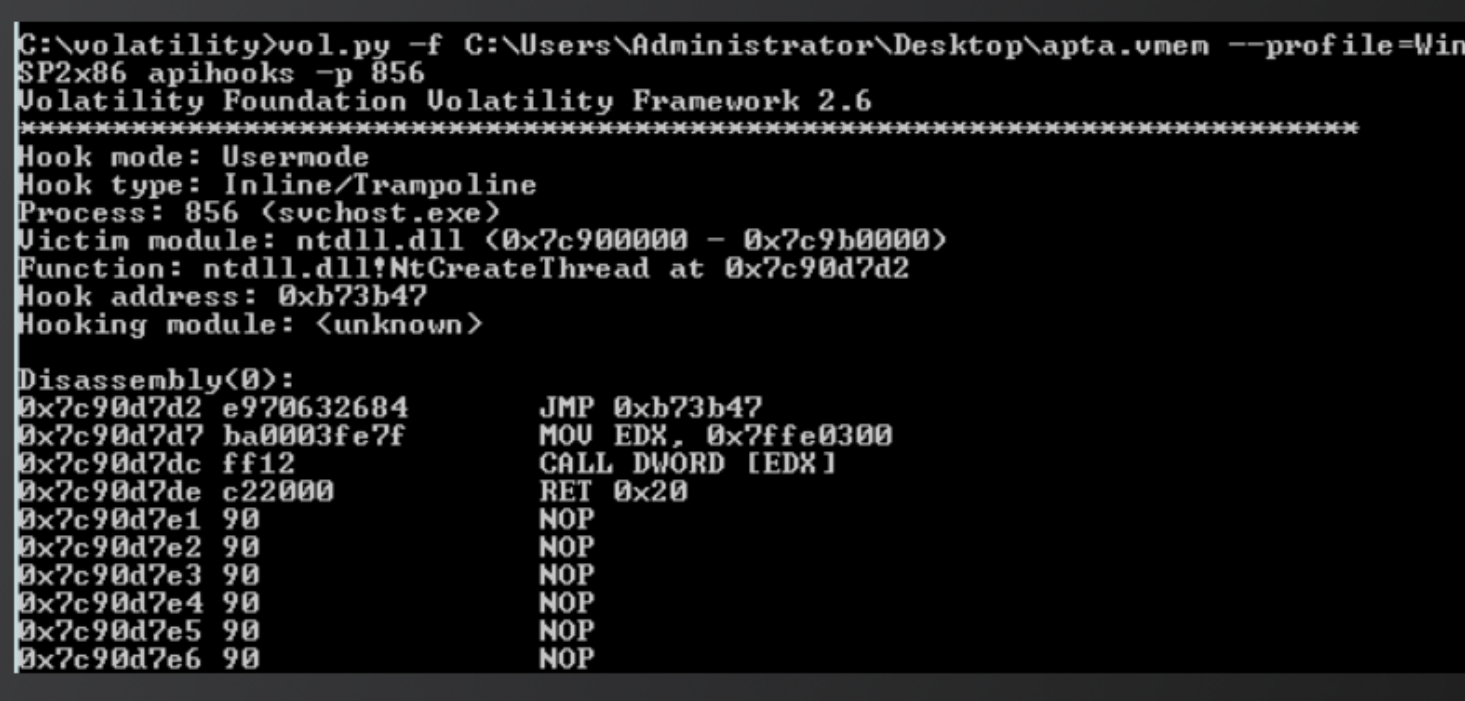
```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86
```

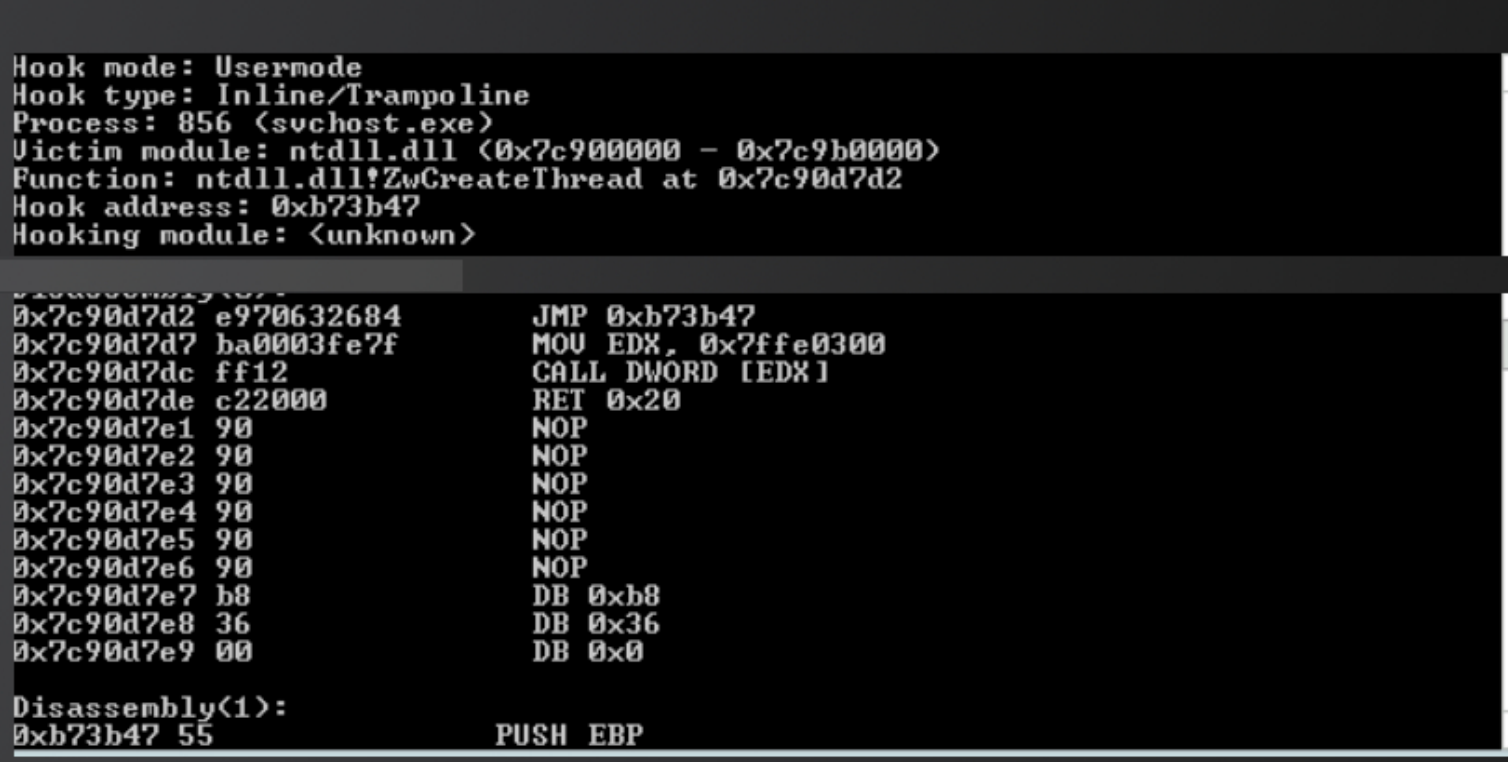
Based on the output we can see 1 memory section containing an MZ header which is an indicator of code injection.


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXP
SP2x86 malfind -p 856
Volatility Foundation Volatility Framework 2.6
Process: svchost.exe Pid: 856 Address: 0xb70000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 38, MemCommit: 1, PrivateMemory: 1, Protection: 6

0x00b70000  4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00   MZ..............
0x00b70010  b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00   ........@.......
0x00b70020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0x00b70030  00 00 00 00 00 00 00 00 00 00 00 00 d0 00 00 00   ................

0x00b70000 4d               DEC EBP
0x00b70001 5a               POP EDX
0x00b70002 90               NOP
0x00b70003 0003             ADD [EBX], AL
0x00b70005 0000             ADD [EAX], AL
0x00b70007 000400           ADD [EAX+EAX], AL
0x00b7000a 0000             ADD [EAX], AL
0x00b7000c ff               DB 0xff
0x00b7000d ff00             INC DWORD [EAX]
0x00b7000f 00b800000000     ADD [EAX+0x0], BH
```

Let's see if this process is hooking to any APIs with **APIHOOKS**.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXPSP2x86
```


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\apta.vmem --profile=WinXP
SP2x86 apihooks -p 856
Volatility Foundation Volatility Framework 2.6
************************************************************************
Hook mode: Usermode
Hook type: Inline/Trampoline
Process: 856 (svchost.exe)
Victim module: ntdll.dll (0x7c900000 - 0x7c9b0000)
Function: ntdll.dll!NtCreateThread at 0x7c90d7d2
Hook address: 0xb73b47
Hooking module: <unknown>

Disassembly(0):
0x7c90d7d2 e970632684       JMP 0xb73b47
0x7c90d7d7 ba0003fe7f       MOV EDX, 0x7ffe0300
0x7c90d7dc ff12             CALL DWORD [EDX]
0x7c90d7de c22000           RET 0x20
0x7c90d7e1 90               NOP
0x7c90d7e2 90               NOP
0x7c90d7e3 90               NOP
0x7c90d7e4 90               NOP
0x7c90d7e5 90               NOP
0x7c90d7e6 90               NOP
```


```
Hook mode: Usermode
Hook type: Inline/Trampoline
Process: 856 (svchost.exe)
Victim module: ntdll.dll (0x7c900000 - 0x7c9b0000)
Function: ntdll.dll!ZwCreateThread at 0x7c90d7d2
Hook address: 0xb73b47
Hooking module: <unknown>

0x7c90d7d2 e970632684       JMP 0xb73b47
0x7c90d7d7 ba0003fe7f       MOV EDX, 0x7ffe0300
0x7c90d7dc ff12             CALL DWORD [EDX]
0x7c90d7de c22000           RET 0x20
0x7c90d7e1 90               NOP
0x7c90d7e2 90               NOP
0x7c90d7e3 90               NOP
0x7c90d7e4 90               NOP
0x7c90d7e5 90               NOP
0x7c90d7e6 90               NOP
0x7c90d7e7 b8               DB 0xb8
0x7c90d7e8 36               DB 0x36
0x7c90d7e9 00               DB 0x0

Disassembly(1):
0xb73b47 55               PUSH EBP
```

From the output we see 2 APIs are hooked (**NtCreateThread and ZwCreateThread**). **Unknown** being shown as the **Hooking module** adds to conclusion suspicious activity.

Note: We have only focused on process 856 but you can use these techniques with nearly all of the processes as they're all injected. The malware you just examined was **Zeus**.

Task 4: Create a MANs file of APTB.VMEM.

Follow the same steps from Task 1 but you're performing the steps against **APTB.VMEM** and naming the analysis file as **APTB**.

1. Click on **Memory Sections** (submenu item under **Processes**) under **Analysis Data**.
2. Click on **Injected Memory Sections** under **Filters**.

We should see 1 item listed as injected with code and it has the same process as the previous memory image. Next we'll look at **Hooks** to see any indicators of some sort of rootkit.

1. Click on **Hooks** under **Analysis Data**.
2. Click on **SSDT** Hooks under **Filters**.
3. Click on **IDT** Hooks under **Filters**.
4. Click on **Untrusted Hooks** under **Filters**.

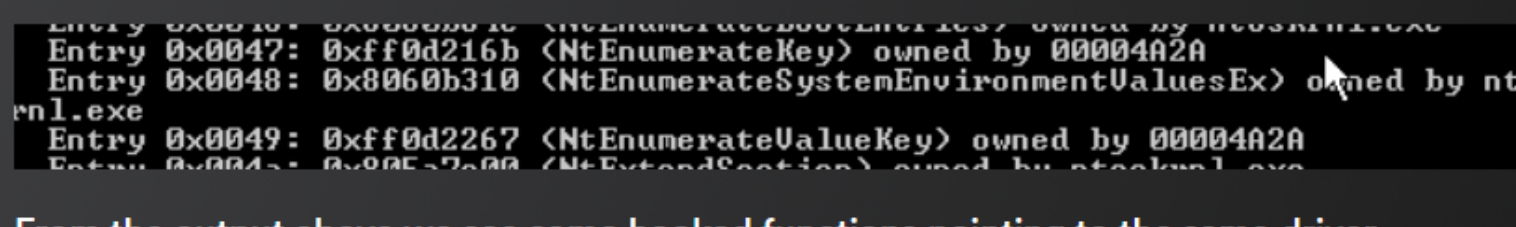
We didn't see anything under SSDT and IDT Hooks but we're seeing some entries under Untrusted Hooks. As before, we can now take a closer look at things in Volatility.

Task 6: Analyze APTB.VMEM within Volatility.

```
# As before we will need to determine what profile to use for this memory image fil
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem imageinfo
```

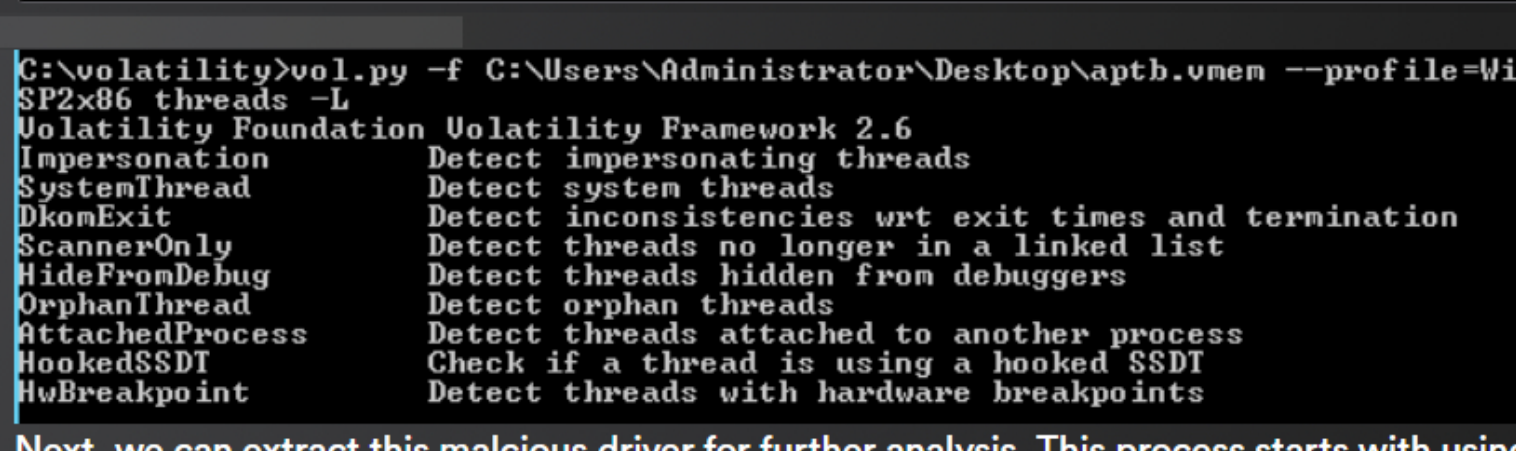
Since the goal is to look for a rootkit, we should only focus on the Volatility plugins for rootkit detection. Let's use **SSDT**.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86
```


```
  Entry 0x0047: 0xff0d216b (NtEnumerateKey) owned by 00004A2A
  Entry 0x0048: 0x8060b310 (NtEnumerateSystemEnvironmentValuesEx) owned by ntosk
rnl.exe
  Entry 0x0049: 0xff0d2267 (NtEnumerateValueKey) owned by 00004A2A
```

From the output above we see some hooked functions pointing to the same driver, **00004A2A.sys.** Alternatively the **THREADS** plugin could be used to obtain the same conclusion about the malicious driver.

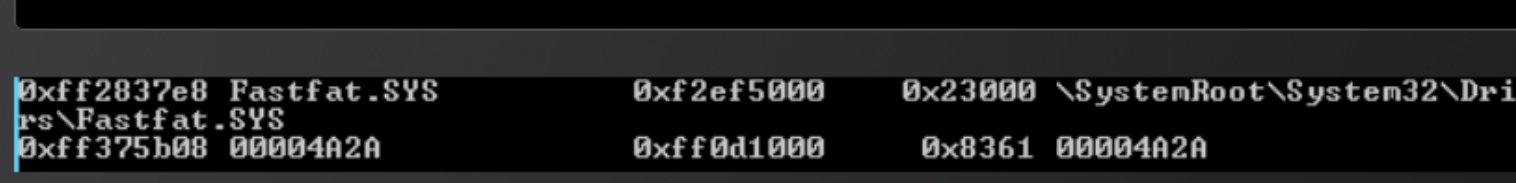
```
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86
```


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXP
SP2x86 threads -L
Volatility Foundation Volatility Framework 2.6
Impersonation        Detect impersonating threads
SystemThread         Detect system threads
DkomExit             Detect inconsistencies wrt exit times and termination
ScannerOnly          Detect threads no longer in a linked list
HideFromDebug        Detect threads hidden from debuggers
OrphanThread         Detect orphan threads
AttachedProcess      Detect threads attached to another process
HookedSSDT           Check if a thread is using a hooked SSDT
HwBreakpoint         Detect threads with hardware breakpoints
```

Next, we can extract this malcious driver for further analysis. This process starts with using **MODULES**

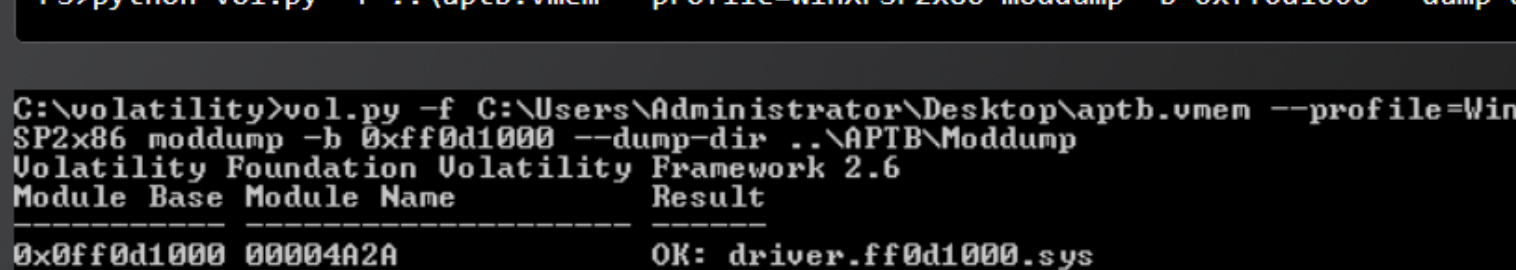
.

```
PS>python vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXPSP2x86
```


```
0xff2837e8 Fastfat.SYS          0xf2ef5000    0x23000 \SystemRoot\System32\Drive
rs\Fastfat.SYS
0xff375b08 00004A2A             0xff0d1000     0x8361 00004A2A
```

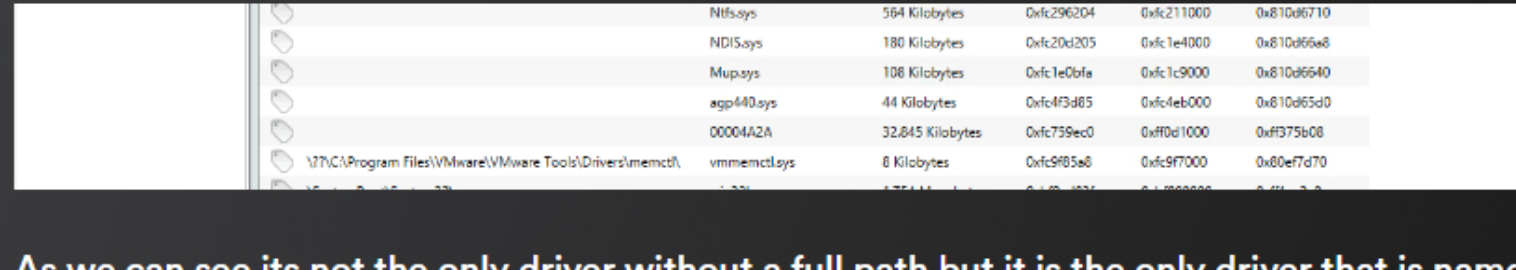
We need to obtain the base address. The base address is **0xff0d1000**. Now we can use **MODDUMP** to extract the driver from the memory image.

```
PS>python vol.py -f ..\aptb.vmem --profile=WinXPSP2x86 moddump -b 0xff0d1000 --dump-dir ..\
```


```
C:\volatility>vol.py -f C:\Users\Administrator\Desktop\aptb.vmem --profile=WinXP
SP2x86 moddump -b 0xff0d1000 --dump-dir ..\APTB\Moddump
Volatility Foundation Volatility Framework 2.6
Module Base Module Name          Result
----------- -------------------- ------
0x0ff0d1000 00004A2A             OK: driver.ff0d1000.sys
```

Let's look for this malicious driver in Redline and see if it would have been easy to identify.

1. Open Redline and load **APTB.MANS** file.
2. Click on **Driver Modules** under **Analysis Data**.
3. Click on the **Name** column to sort the column contents.


	Ntfs.sys	564 Kilobytes	0xfc296204	0xfc211000	0x810d6710
	NDIS.sys	180 Kilobytes	0xfc20d205	0xfc1e4000	0x810d66a8
	Mup.sys	108 Kilobytes	0xfc1e0bfa	0xfc1c9000	0x810d6640
	agp440.sys	44 Kilobytes	0xfc4f3d85	0xfc4eb000	0x810d65d0
	00004A2A	32,845 Kilobytes	0xfc759ec0	0xff0d1000	0xff375b08
\??\C:\Program Files\VMware\VMware Tools\Drivers\memctl\	vmmemctl.sys	8 Kilobytes	0xfc9f85a8	0xfc9f7000	0x80ef7d70

As we can see its not the only driver without a full path but it is the only driver that is named with a number value. Also you can see the Module Base address which was obtained in Volatility using **MODULES** and extracted using **MODDUMP**.

Note: The malware you just analyzed was **Black Energy**.

Conclusion

After this lab you will be able to Use Redline to find evidence of code injection and/or rootkits, Use Volatility to find evidence of code injection and/or rootkits, Use Volatility to find information that was not seen in Redline.